

REPORTE DE PRÁCTICA: GESTIÓN DE ARCHIVOS Y MODELO DAC EN GNU/LINUX

Asignatura: Sistemas Operativos

Tema: Permisos UNIX (DAC), Inodos y Semántica de Llamadas al Sistema

1. RESUMEN Y OBJETIVOS

El presente laboratorio analiza el comportamiento del sistema de archivos en GNU/Linux frente a operaciones de creación, copia (cp) y eliminación (rm) de archivos y directorios bajo distintas identidades de usuario. El objetivo central es contrastar los permisos intrínsecos de un archivo regular con los permisos del directorio contenedor, justificando teóricamente la mutación de metadatos y la eliminación de archivos protegidos.

2. REGISTRO EXPERIMENTAL SINTÉTICO

Etapa	Comandos Clave	Evento / Salida Observable
1. Inicialización y Escalado	mkdir practica1 practica2 cd practica1 sudo nano Readme.txt	Directorios creados con modo 0775 (drwxrwxr-x). Readme.txt queda con propietario root:root y permisos -rw-r--r-- (0644).
2. Transferencia y Estructuración	cp -v Readme.txt ~/Documentos/practica2/ mkdir vacia info sudo nano info/Readme2.txt cp -v -r vacia info ../practica1	En practica2, la copia de Readme.txt pasa a ser propiedad de teemoo:teemoo. La copia recursiva replica la jerarquía de subdirectorios hacia practica1.
3. Destrucción de	rm Readme.txt	rm solicita confirmación

Archivos	(confirmación: s) rm -r -v info/	interactiva por falta de permiso de escritura, pero permite el borrado sin sudo. rm -r elimina recursivamente el contenido interno y luego el directorio.
----------	---	---

3. ANÁLISIS REFLEXIVO Y JUSTIFICACIÓN TEÓRICA

3.1. Permisos Base y el Rol de umask

Al invocar mkdir, los directorios heredan los permisos determinados por el modo base (**0777**) atenuado por la máscara de usuario:

$$\text{Modo final} = 0777 \wedge (\sim 0002) = 0775 \quad (\text{drwxrwxr-x})$$

El tamaño reportado de **4096 bytes** corresponde al bloque mínimo de asignación física del sistema de archivos.

3.2. Transmutación de Propiedad en la Copia (cp)

- **Observación:** El archivo original en practica1 pertenecía a root, pero su copia en practica2 pasó a pertenecer a teemoo.
- **Justificación:** cp no traslada inodos existentes; realiza una lectura del origen (open(..., O_RDONLY)) y crea un **nuevo archivo e inodo** en el destino mediante open(..., O_CREAT|O_WRONLY). Dado que la llamada al sistema la emite el proceso de teemoo (**EUID $\neq$ 0**), el kernel asigna automáticamente al usuario invocador como dueño del nuevo inodo. Para preservar la titularidad original se requería cp -p con privilegios de administrador.

3.3. La Paradoja de la Eliminación de Archivos Protegidos (rm)

- **Observación:** El usuario ordinario teemoo pudo borrar practica1/Readme.txt (propiedad de root y sin bit **w** para otros), respondiendo afirmativamente a la advertencia:
rm: ¿borrar el regular file 'Readme.txt' protegido contra escritura? (s/n) s
- **Justificación a nivel de Kernel:**

1. **Desvinculación vs. Modificación:** Eliminar un archivo no altera su contenido ni su inodo; retira la entrada de directorio (*dentry*) mediante la llamada al sistema:

`unlink(pathname)`

2. **Validación de Permisos:** El kernel valida los permisos sobre el **directorio contenedor**, no sobre el archivo objeto. Puesto que teemoo posee permiso de escritura (`w`) y paso (`x`) sobre practica1/, tiene autorización para desvincular cualquier entrada contenida en él.
3. **Naturaleza del Prompt:** La advertencia interactiva no es una restricción del sistema operativo, sino una medida de seguridad implementada en el espacio de usuario por el binario `rm` al detectar la ausencia del bit `w` en el archivo regular.

3.4. Eliminación Recursiva de Directorios (`rm -r`)

Un directorio no puede desvincularse si contiene entradas activas. La bandera `-r` aplica un recorrido en post-orden: invoca primero `unlink()` sobre cada nodo hoja (`info/Readme2.txt`) y, una vez vacía la tabla de entradas, libera el directorio padre mediante la llamada `rmdir()`.

4. CONCLUSIONES

1. **Independencia Inodo-Contenedor:** La permanencia de un archivo depende estrictamente de los privilegios del directorio que lo aloja, demostrando que poseer derecho de escritura en el contenedor otorga control sobre la existencia de sus elementos.
2. **Ciclo de Vida de E/S Estándar:** Las operaciones convencionales de creación y duplicación operan bajo el contexto del proceso invocador, alterando la metadata si no se fuerza explícitamente su persistencia.
3. **Mecanismos de Protección:** El sistema operativo separa la política de seguridad estricta (DAC en el kernel vía `unlink`) de las salvaguardas de usabilidad preventiva (heurística interactiva de `rm` en el shell).